

Graph Database and Cypher Queries

CITS3401 Data Warehousing Project 2
24006703 Raynard Alessandro Djauhari

Table of Contents

Part 1: Introduction and Database Design

- 1.1 Introduction
- 1.2 Graph model description
- 1.3 Arrows app screenshot
- 1.4 Design choices

Part 2: ETL Process and Graph Creation

- 2.1 Dataset Overview
- 2.2 Dataset Issues
- 2.3 Data Cleaning
- 2.4 Data Transformation
- 2.5 Final CSVs
- 2.6 Import process (Cypher)
- 2.7 Neo4j Aura Database Information

Part 3: Cypher Queries

- 3.1 Query A: Australian Airlines
- 3.2 Query B: Domestic vs International Routes
- 3.3 Query C: Most Served Airport Pair
- 3.4 Query D: Top 5 Airport Pairs by Distinct Aircraft
- 3.5 Query E: Routes from Beijing to Perth

Part 4: Self-Designed Queries

- 4.1 BFS shortest path query
- 4.2 APOC meta schema query

Part 5: Discussions

- 5.1 Performance Analysis
- 5.2 Performance Discussion
- 5.3 Graph Data Science Applications
- 5.4 Graph Data Science Discussion
- 5.5 Importance in schema design
- 5.6 Leveraging metadata for effective Cypher queries

Part 6: Appendix

- 6.1 AI Declaration
- 6.2 References

Part 1: Introduction and Database Design

1.1 Introduction

This project investigates the application of graph database technology to a real-world aviation dataset. A property graph model was designed using the Arrows app and implemented in Neo4j AuraDB. The ETL process involved transforming the raw dataset into structured CSV files, which were then imported into Neo4j using Cypher. A series of queries were developed to answer business questions relating to route classification, airport connectivity, and aircraft diversity, with additional self-designed queries demonstrating further analytical capabilities.

1.2 Graph Model Description

The property graph consists of four node types and four relationship types. Nodes represent the core entities in the dataset: **Airline**, **Airport**, **Route**, **Aircraft**. Relationships capture how these entities interact:

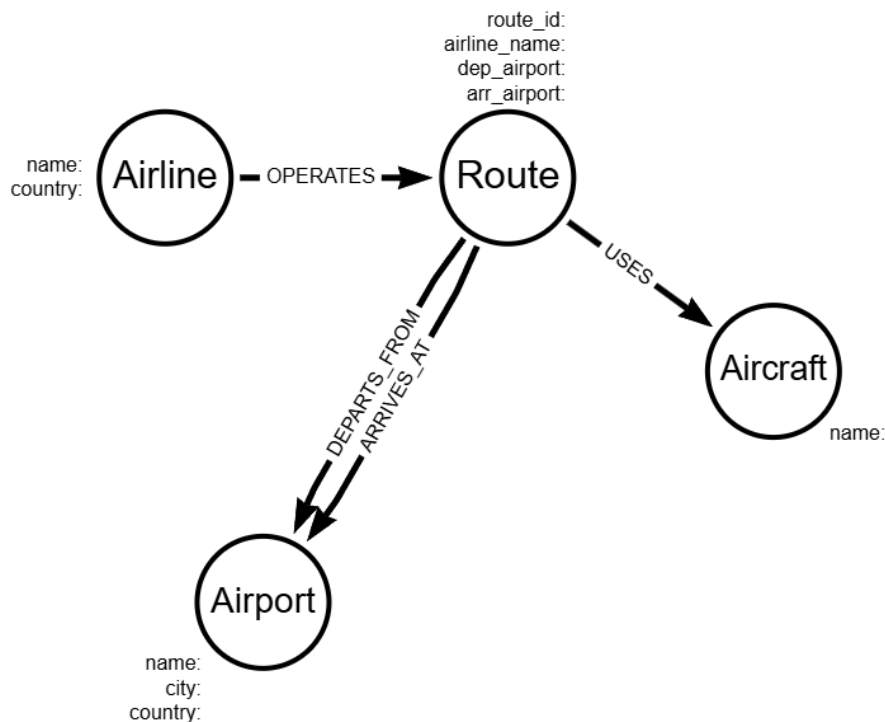
Relationship	From	To
OPERATES	Airline	Route
DEPARTS_FROM	Route	Airport
ARRIVES_AT	Route	Airport
USES	Route	Aircraft

Relationships carry no properties; all relevant data is stored on nodes.

Each node carries the following properties:

Node	Properties
Airline	name, country
Airport	name, city, country
Route	route_id, airline_name, dep_airport, arr_airport
Aircraft	name

1.3 Arrows app screenshot



1.4 Design Choices

a. Graph model reasoning

I designed the graph model around the structure of the raw dataset, where each row represents a service operated by a specific airline between two airports using specific aircraft. This naturally maps to a **Route** node as the central entity, acting as the unique identifier for each (**airline**, **departure airport**, **arrival airport**) combination. All relationship properties are empty; relevant data is stored on nodes, keeping the model normalised.

b. Route-as-node vs Route-as-relationship

There are two competing models for representing routes in this dataset.

```
(:Airport)-[:ROUTE]->(:Airport)
```

The first model represents **Route** as a direct relationship between **Airport** nodes, with **airline** and **aircraft** stored as properties or via additional edges. The unit coordinator noted this is the more natural model for path queries such as Q3e, which asks for paths using at most 3 route hops. Variable-length traversal is simpler since each hop is a single relationship.

```
(:Airline)-[:OPERATES]->(:Route)-[:DEPARTS_FROM|ARRIVES_AT]->(:Airport)
```

The second model represents **Route** as a node, with **Airline**, **Airport**, and **Aircraft** connected to it via relationships. This is the model I implemented.

I opted for the **Route-as-node** model for two reasons. First, the many-to-many relationship between routes and aircraft cannot be cleanly modelled if **Route** is a relationship, since Neo4j relationships cannot connect to other nodes. Storing aircraft as a list property on the relationship would prevent them from being queried as distinct entities, which is a hard requirement for Q3d's `count(DISTINCT ac.name)`. Second, attributing an **Airline** to a **Route** is only possible if **Route** is a node. A **Route-as-relationship** model would have to store the airline as a property, losing the ability to traverse from **Airline** to **Route** in the graph.

The tradeoff is that Q3e becomes more complex. In the Route-as-node model, each logical hop between two airports requires traversing two relationships (**DEPARTS_FROM** then **ARRIVES_AT**), so a 3-hop limit requires a path bound. More on this in section [5.2 Performance Discussion](#).

Other design choices

Route connectivity is inherently relational. Airports connect to other airports via routes, making graph traversal a natural fit for path-finding queries, which would be significantly more complex to express in SQL.

The same Airport node serves dual roles as both departure and arrival endpoint, distinguished purely by relationship direction (**DEPARTS_FROM** vs **ARRIVES_AT**). This avoids data duplication and keeps the model normalised.

Limitations

The **Route** node stores **airline_name**, **dep_airport**, and **arr_airport** as redundant properties alongside the relationships. This was included for query convenience but introduces some denormalisation. The model does not capture passenger-level data, flight schedules, or pricing, limiting the analytical scope to route and aircraft-level queries.

Part 2: ETL Process and Graph Creation

2.1 Dataset Overview

The raw dataset contains 57,301 records sourced from the TU Graz ISDS aviation repository [1], preprocessed by combining multiple CSV files and removing irrelevant columns. Each record represents a flight service operated by a specific airline between two airports.

The dataset includes nine fields: airline name, airline country, departure airport name, departure airport city, departure airport country/region, arrival airport name, arrival airport city, arrival airport country/region, and aircraft types.

2.2 Dataset Issues

The raw dataset was systematically inspected for data quality issues using a dedicated audit script (`flag.py`). The script runs nine independent checks each listed below. Issues fall into two categories: data quality errors that must be corrected before ETL (handled by `clean.py`, section 2.3), and structural format issues that are resolved during ETL (handled by `etl.py`, section 2.4).

There are the findings of `flag.py`:

a. Stray uppercase city

One city value, `BRISTOL`, appears in all caps in 6 rows for Tri-Cities Regional TN/VA Airport, while every other city in the dataset uses title case. Normalised to `Bristol` in `clean.py`.

b. Airport country errors

29 airport names appear in the raw data with the same (name, city) listed under different countries. For example, `London Heathrow Airport` appears as both United Kingdom and Canada. Each case was manually verified against public sources and reassigned to the correct country in `clean.py`.

c. Airport name ambiguity

11 distinct airports have the same name. For example, `Albany Airport` exists in `Albany, Australia` and `Albany, United States`. This is not a data error as I checked each of them and found that they all exist and are distinct.

d. Self-loop route

One case of a self-loop: `South Pacific Island Airways` operating `Iskandar Airport, Pangkalan Bun, Indonesia` to itself. This is the only row I dropped.

e. Duplicate route records

4 (`airline, dep_air, arr_air`) combinations appear in multiple rows with different aircraft lists. Treating these as separate routes would result in duplicate `Route` nodes. Resolved in `etl.py` by grouping on (`airline, dep, arr`) so there are no duplicates.

f. Semicolon-delimited aircraft field

The plane name column stores multiple values in a single field delimited by semicolons, for example `Harbin Y12; DHC-6 Twin Otter; BN-2A Islander`. 13,673 rows contain more than one aircraft, with up to 9 aircraft per row. Resolved in `etl.py` by splitting the field on `;` and emitting one (`route, aircraft`) pair per element into `rel Uses.csv`.

g. Other Checks

These checks show no issue: null values, trailing whitespaces, airline-country consistency

2.3 Data Cleaning

The data cleaning was implemented using the `pandas` library in `clean.py`. It resolves (a) (b) and (d) from section 2.2 and outputs `cleaned.csv` which is later used on by `etl.py`

a. Load raw dataset

```
df = pd.read_csv("raw.csv")
df.columns = [
    "airline_name", "airline_country",
    "dep_airport", "dep_city", "dep_country",
    "arr_airport", "arr_city", "arr_country",
    "planes"
]
```

b. Normalize stray uppercase city

The city value `BRISTOL` is normalized to `Bristol` to match the title case convention used throughout the rest of the dataset.

```
df.loc[df["dep_city"] == "BRISTOL", "dep_city"] = "Bristol"
df.loc[df["arr_city"] == "BRISTOL", "arr_city"] = "Bristol"
```

c. Correct airport country errors

A dictionary maps each erroneous (`airport_name`, `city`) pair to its canonical country, verified manually against public sources. A helper function applies the correction to both the departure and arrival sides of every row.

```
COUNTRY_FIXES = {
    ("London Heathrow Airport", "London"): "United Kingdom",
    ("Sydney Kingsford Smith International Airport", "Sydney"): "Australia",
    # ... 28 additional entries
}

def fix_country(name, city, country):
    return COUNTRY_FIXES.get((name, city), country)

df["dep_country"] = df.apply(
    lambda r: fix_country(r["dep_airport"], r["dep_city"], r["dep_country"]), axis=1)
df["arr_country"] = df.apply(
    lambda r: fix_country(r["arr_airport"], r["arr_city"], r["arr_country"]), axis=1)
```

d. Drop self-loop

The line below handles all self-loop removal. There was only 1 self-loop so only 1 row was dropped.

```
df = df[df["dep_airport"] != df["arr_airport"]].reset_index(drop=True)
```

e. Output

`cleaned.csv` is written with identical column names and structure to `raw.csv`, producing 57,300 rows (57,301 minus the one dropped self-loop).

```
df.to_csv("cleaned.csv", index=False)
```

2.4 Data Transformation

The data transformation is done in `etl.py` using `cleaned.csv` and transforms the raw dataset into 8 CSV files suitable for direct import into Neo4j via `LOAD CSV`. It also resolves (e) and (f) from section 2.2.

a. Load dataset

The cleaned dataset is loaded into a `pandas DataFrame`. At this point it contains 57,300 rows and 9 columns with the original column names from the source CSV.

```
df = pd.read_csv("cleaned.csv") # line 3
```

b. Rename columns

Columns are renamed to standardised identifiers to simplify downstream references throughout the script.

```
df.columns = [  
    "airline_name", "airline_country",  
    "dep_airport", "dep_city", "dep_country",  
    "arr_airport", "arr_city", "arr_country",  
    "planes"  
] # lines 5-10
```

c. Define aircraft union function

This function resolves the semicolon-separated aircraft issue identified in Section 2.2. For a group of rows sharing the same (airline, departure, arrival), it splits each aircraft string on `“;”`, strips whitespace, and collects all unique aircraft names into a set. The result is returned as a single sorted semicolon-delimited string representing the union of all aircraft for that route.

```
def union_planes(series):  
    all_planes = set()  
    for val in series:  
        if pd.notna(val):  
            for p in val.split(";"):  
                p = p.strip()  
                if p:  
                    all_planes.add(p)  
    return ";".join(sorted(all_planes)) # lines 13-21
```

d. Deduplicate routes

Records are grouped by the combination of `(airline_name, dep_airport, arr_airport)`, which is the natural unique identifier for a route. All other fields are taken from the first occurrence, and the `planes` field is aggregated using `union_planes`. This collapses the 4 duplicate rows identified in Section 2.2, reducing the dataset from 57,300 to 57,296 rows.

```
routes = df.groupby(  
    ["airline_name", "dep_airport", "arr_airport"],  
    as_index=False  
) .agg(  
    airline_country=("airline_country", "first"),  
    dep_city=("dep_city", "first"),  
    dep_country=("dep_country", "first"),  
    arr_city=("arr_city", "first"),  
    arr_country=("arr_country", "first"),  
    planes=("planes", union_planes)  
) # lines 23-33
```

e. Assign route IDs

A sequential integer `route_id` is assigned to each deduplicated route. This serves as the primary key for the `Route` node in Neo4j.

```
routes.insert(0, "route_id", range(1, len(routes) + 1)) # line 35
```

f. Generate node CSVs

`airlines.csv` is produced by selecting and deduplicating the airline columns, yielding 488 rows.

```
airlines = routes[["airline_name", "airline_country"]].drop_duplicates() # line 38
airlines.to_csv("airlines.csv", index=False)
```

```
dep = routes[["dep_airport", "dep_city", "dep_country"]].rename(...)
arr = routes[["arr_airport", "arr_city", "arr_country"]].rename(...)
airports = pd.concat([dep, arr]).drop_duplicates(subset=["airport_name", "city", "country"])
airports.to_csv("airports.csv", index=False) # lines 43-47
```

`airports.csv` is produced by extracting departure and arrival airport columns separately, renaming them to a common schema, concatenating both sets, and deduplicating on (`airport_name`, `city`, `country`). This handles the fact that the same airport appears in both departure and arrival columns across different rows, yielding 2,806 unique airports.

```
aircraft = pd.DataFrame(sorted(all_planes), columns=["aircraft_name"])
aircraft.to_csv("aircraft.csv", index=False)
```

All aircraft names are collected into a set by iterating over the unified `planes` field and splitting on `;`, producing 114 distinct aircraft types in `aircraft.csv`.

```
routes[["route_id", "dep_airport", "arr_airport", "airline_name"]].to_csv("routes.csv")
```

`routes.csv` contains one row per deduplicated route with its assigned `route_id`, yielding 57,296 rows.

g. Generate relationship CSVs

Three relationship CSVs are generated directly from the routes `DataFrame`, each mapping a `route_id` to its corresponding airline or airport by name.

```
routes[["airline_name", "route_id"]].to_csv("rel_operates.csv") # line 67
routes[["route_id", "dep_airport", "dep_city", "dep_country"]].to_csv("rel_departs.csv")
routes[["route_id", "arr_airport", "arr_city", "arr_country"]].to_csv("rel_arrives.csv")
```

`rel_uses.csv` is generated by iterating over every route and exploding the semicolon-delimited `planes` field into individual rows, one per aircraft type. This produces 76,967 route-aircraft pairs, correctly representing the many-to-many relationship between routes and aircraft in the graph.

```
for _, row in routes.iterrows():
    for p in row["planes"].split(";"):
        p = p.strip()
        if p:
            uses_rows.append({"route_id": row["route_id"], "aircraft_name": p})
rel_uses = pd.DataFrame(uses_rows)
rel_uses.to_csv("rel_uses.csv", index=False) # lines 80-86
```

2.5 Final CSVs

The ETL script produces 8 CSV files split into node CSVs and relationship CSVs. All files were uploaded to a public GitHub repository and accessed via raw URLs during the Neo4j import process.

File	Type	Rows	Purpose
airlines.csv	Node	488	One row per distinct airline
airports.csv	Node	2,806	One row per distinct airport
aircraft.csv	Node	114	One row per distinct aircraft type
routes.csv	Node	57,296	One row per deduplicated route
rel_operates.csv	Relationship	57,296	Maps airline to route
rel_departs.csv	Relationship	57,296	Maps route to departure airport
rel_arrives.csv	Relationship	57,296	Maps route to arrival airport
rel_uses.csv	Relationship	76,967	Maps route to aircraft (exploded)

2.6 Import Process (Cypher)

All 8 CSVs were imported into Neo4j AuraDB using `LOAD CSV` Cypher queries, stored in `import.cypher`. The import followed this order: constraints first, then node CSVs, then relationship CSVs. This ordering is critical as relationship imports use `MATCH` to find existing nodes, so all nodes must exist before relationships are created.

Constraints were created first to enforce uniqueness and prevent duplicate nodes during import:

```
CREATE CONSTRAINT route_id IF NOT EXISTS
FOR (r:Route) REQUIRE r.route_id IS UNIQUE;
CREATE CONSTRAINT airport_identity IF NOT EXISTS
FOR (a:Airport) REQUIRE (a.name, a.city, a.country) IS UNIQUE;
```

Each node CSV was then imported using `MERGE`, which creates a node only if it does not exist yet. Relationship CSVs were imported last, matching on the unique identifiers established by the constraints. CSV files were referenced via their raw GitHub URLs, as Neo4j AuraDB cannot access local files directly.

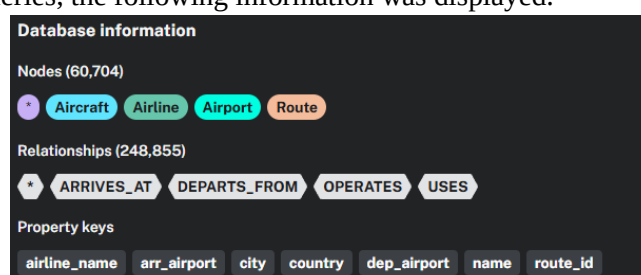
Below are snippets for node import and relationship import.

```
FROM 'https://raw.githubusercontent.com/IIEnat/Data-Warehousing-2/refs/heads/main/csv/airlines.csv' AS row
MERGE (a:Airline {name: row.airline_name})
SET a.country = row.airline_country;
```

```
FROM 'https://raw.githubusercontent.com/IIEnat/Data-Warehousing-2/refs/heads/main/csv/rel_departs.csv' AS row
MATCH (r:Route {route_id: toInteger(row.route_id)})
MATCH (a:Airport {name: row.dep_airport, city: row.dep_city, country: row.dep_country})
MERGE (r)-[:DEPARTS_FROM]->(a);
```

2.7 Neo4j Aura Database Information

After running the cypher queries, the following information was displayed.



Part 3: Cypher Queries

3.1 Query A: Australian Airlines

This query retrieves all distinct airline names where the airline's country is Australia. It directly traverses **Airline** nodes and filters on the **country** property, requiring no relationship traversal since airline country is stored as a node property.

```
MATCH (a:Airline)
WHERE a.country = 'Australia'
RETURN DISTINCT a.name AS airline_name;
```

The result shows that there are only 3 airlines based in Australia.



The screenshot shows a query editor with the Cypher query: `MATCH (a:Airline) WHERE a.country = 'Australia'`. The results are displayed in a table view with the column `airline_name`. Three records are shown: "Transaustralian Air Express", "Transpac Express", and "Whyalla Airlines". A status bar at the bottom indicates "Started streaming 3 records after 15 ms and completed after 16 ms."

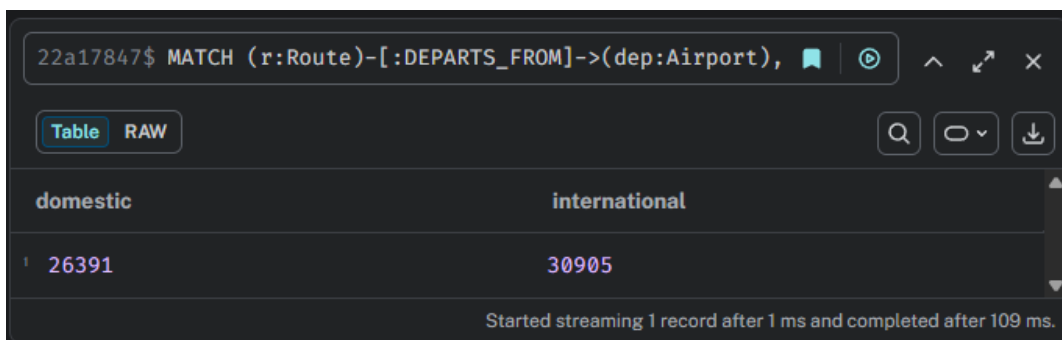
airline_name
1 "Transaustralian Air Express"
2 "Transpac Express"
3 "Whyalla Airlines"

3.2 Query B: Domestic vs International Routes

This query classifies all route records as either domestic or international. A route is domestic if its departure and arrival airports share the same **country** property, and international otherwise. The query traverses each **Route** node to both its departure and arrival **Airport** nodes via **DEPARTS_FROM** and **ARRIVES_AT** relationships, then uses conditional aggregation to produce a single summary row.

```
MATCH (r:Route)-[:DEPARTS_FROM]->(dep:Airport), (r)-[:ARRIVES_AT]->(arr:Airport)
RETURN
  SUM(CASE WHEN dep.country = arr.country THEN 1 ELSE 0 END) AS domestic,
  SUM(CASE WHEN dep.country <> arr.country THEN 1 ELSE 0 END) AS international;
```

The result shows that of 57,296 routes, 26,391 (46%) are domestic while 30,905 (54%) are international.



The screenshot shows a query editor with the Cypher query: `MATCH (r:Route)-[:DEPARTS_FROM]->(dep:Airport), (r)-[:ARRIVES_AT]->(arr:Airport)`. The results are displayed in a table view with two columns: `domestic` and `international`. A single record is shown with values 26391 and 30905. A status bar at the bottom indicates "Started streaming 1 record after 1 ms and completed after 109 ms."

domestic	international
1 26391	30905

3.3 Query C: Most Served Airport Pair

This query finds the airport pair with the greatest number of route records, treating $A \rightarrow B$ and $B \rightarrow A$ as the same pair. To handle directionality, a canonical ordering is applied using a **CASE** expression so that the airport whose name comes first alphabetically is always assigned as **airport1**. This ensures that regardless of which direction a route is stored, both directions map to the same pair. The **WITH** clause is used as required by the question to pass the aggregated pair and count to the final **RETURN**.

```
MATCH (r:Route)-[:DEPARTS_FROM]->(dep:Airport), (r)-[:ARRIVES_AT]->(arr:Airport)
WITH
  CASE WHEN dep.name < arr.name THEN dep.name ELSE arr.name END AS airport1,
  CASE WHEN dep.name < arr.name THEN arr.name ELSE dep.name END AS airport2
WITH airport1, airport2, count(*) AS route_count
ORDER BY route_count DESC
LIMIT 1
RETURN airport1, airport2, route_count;
```

The results show that the Chicago O'Hare International Airport and the Hartsfield Jackson Atlanta International Airport route is served by the highest number of airlines.



airport1	airport2	route_count
"Chicago O'Hare International Airport"	"Hartsfield Jackson Atlanta International Airport"	37

3.4 Query D: Top 5 Airport Pairs by Distinct Aircraft

This query finds the top 5 airport pairs served by the greatest number of distinct aircraft types across all routes and airlines. As with Query C, a canonical alphabetical ordering is applied to treat $A \rightarrow B$ and $B \rightarrow A$ as the same pair. The query traverses each **Route** node to its departure and arrival airports via **DEPARTS_FROM** and **ARRIVES_AT**, and to its aircraft via **USES**. **count(DISTINCT ac.name)** ensures that the same aircraft type is not double-counted even if it appears on multiple routes between the same pair.

```
MATCH (r:Route)-[:DEPARTS_FROM]->(dep:Airport), (r)-[:ARRIVES_AT]->(arr:Airport), (r)-[:USES]->(ac:Aircraft)
WITH
  CASE WHEN dep.name < arr.name THEN dep.name ELSE arr.name END AS airport1,
  CASE WHEN dep.name < arr.name THEN arr.name ELSE dep.name END AS airport2,
  ac
WITH airport1, airport2, count(DISTINCT ac.name) AS distinct_aircraft
ORDER BY distinct_aircraft DESC
LIMIT 5
RETURN airport1, airport2, distinct_aircraft;
```

The result shows that the Hong Kong International Airport to Incheon International Airport route is served with the highest number of distinct aircrafts at 15. The second highest is tied at 13 and the third highest is tied at 12.

22a17847\$ MATCH (r:Route)-[:DEPARTS_FROM]->(dep:Airport)

Table RAW

	airport1	airport2	distinct_aircraft
1	"Hong Kong International Airport"	"Incheon International Airport"	15
2	"Beijing Capital International Airport"	"Guangzhou Baiyun International Airport"	13
3	"Hong Kong International Airport"	"Suvarnabhumi Airport"	13
4	"Beijing Capital International Airport"	"Shanghai Hongqiao International Airport"	12
5	"Incheon International Airport"	"Kansai International Airport"	12

Started streaming 5 records after 46 ms and completed after 466 ms.

3.5 Query E: Routes from Beijing to Perth

This query finds all distinct travel routes from Beijing Capital International Airport to Perth International Airport where at most 3 hops are traversed. A hop is defined as a single **Route** relationship, meaning at most 3 intermediate flights. The query uses a variable-length path pattern traversing **DEPARTS_FROM** and **ARRIVES_AT** relationships bidirectionally, with a path length bound of 2 to 6 relationship traversals (each hop requires 2 relationship traversals in the graph model). The sequence of airports visited is extracted from the path and deduplicated to treat each unique sequence as a distinct route.

```
MATCH (beijing:Airport {name: 'Beijing Capital International Airport'}),
      (perth:Airport {name: 'Perth International Airport'})
MATCH path = (beijing)-[:DEPARTS_FROM|ARRIVES_AT*2..6]-(perth)
WITH [n IN nodes(path) WHERE n:Airport] AS airports
WITH DISTINCT airports
RETURN count(*) AS distinct_routes;
```

The result shows that there are 653 distinct routes.

22a17847\$ MATCH (beijing:Airport {name: 'Beijing Capital International Airport'}), (perth:Airport {name: 'Perth International Airport'})

Table RAW

	distinct_routes
1	653

Started streaming 1 record after 1 ms and completed after 17,141 ms.

> ⓘ 03N90: Cartesian product

Part 4: Self-Designed Queries

4.1 BFS shortest path query

Question

Given the scale of the aviation network in this dataset, which is 2,806 airports and 57,296 routes, a natural and practically relevant question is: what is the minimum number of flights required to travel from Perth International Airport to any other country?

Justification

This is interesting because it helps people here plan a trip. Countries with a hop of 1 are more accessible and therefore cheaper and more appealing to go to as there is less hassle with the flight industry. Alternatively, countries with a higher hop count are harder to reach from Perth, implying fewer direct airline options, longer travel times, and typically higher ticket prices due to the lack of direct competition on those routes.

From a business perspective, this query also highlights potential gaps in the aviation market as countries requiring 3 or more hops from Perth may represent underserved routes where new direct services could be commercially viable.

Query

```
MATCH (start:Airport {name: 'Perth International Airport'}), (end:Airport)
WHERE end.country <> 'Australia' AND end <> start
MATCH path = shortestPath((start)-[:DEPARTS_FROM|ARRIVES_AT*]-(end))
WITH end.country AS country, min(length(path)/2) AS min_hops
RETURN country, min_hops
ORDER BY min_hops ASC;
```

Explanation

The query uses Neo4j's built-in `shortestPath` function, which performs a BFS traversal from the source node to find the minimum-length path to each target airport. Relationships are traversed bidirectionally across both `DEPARTS_FROM` and `ARRIVES_AT` since a hop between two airports requires crossing both relationship types in the graph model. The path length is divided by 2 to convert from relationship count to hop count. Results are grouped by country, retaining only the minimum hop count across all airports in that country.

Result

The results show that 12 countries are accessible with a direct flight, 109 countries with 2 flights, 93 countries with 3 flights and 8 countries with 4 flights. The snippet below shows all the countries that require at least 4 flights. For the full result see [Q4a_result.csv](#) in the submitted files.

```
Dominica,4
British Virgin Islands,4
Anguilla,4
Saint Kitts and Nevis,4
Venezuela,4
Tuvalu,4
Falkland Islands,4
Saint Vincent and the Grenadines,4
```

4.2 APOC meta schema query

Question

When working with any graph database, a common question often pops up. What does the schema look like after import? This is useful for verifying that the import was successful, confirming that all node labels, relationship types, and properties are present as expected, and understanding the structure of the graph before writing analytical queries. Rather than manually inspecting individual nodes, the APOC library provides a procedure that introspects the entire graph schema programmatically [3].

Query

```
CALL apoc.meta.schema()
YIELD value
RETURN value;
```

Explanation

`apoc.meta.schema()` is a built-in APOC procedure that scans the graph and returns a map describing all node labels, their properties and types, and the relationship types connecting them. It provides a high-level structural overview of the entire database in a single call without requiring any manual traversal. This is particularly useful in the context of this project as it confirms that all 4 node types (`Airline`, `Airport`, `Aircraft`, `Route`) and all 4 relationship types (`OPERATES`, `ARRIVES_AT`, `DEPARTS_FROM`, `USES`) were correctly created during the import process described in Part 2. The output also ties directly into the metadata discussion in Section 5.5, as `apoc.meta.schema()` is itself a metadata retrieval tool as it does not return data from the graph, but rather data about the graph.

Result

```
1  {
2    "value": {
3      "Airline": {
4        "count": 488,
5        "relationships": {
6          "OPERATES": {
7            "count": 0,
8            "properties": {},
9            "direction": "out",
10           "labels": [
11             "Route"
12           ]
13         },
14       },
15       "type": "node",
16       "properties": {
17         "name": {
18           "existence": false,
19           "type": "STRING",
20           "indexed": true,
21           "unique": true
22         },
23         "country": {
24           "existence": false,
25           "type": "STRING",
26           "indexed": false,
27           "unique": false
28         }
29       },
30       "labels": [
31         "Airline"
32       ]
33     },
34     "Airport": {
35       "count": 2795,
36       "relationships": {
37         "ARRIVES_AT": {
38           "count": 1081,
39           "properties": {},
40           "direction": "in",
41           "labels": [
42             "Route"
43           ]
44         },
45         "DEPARTS_FROM": {
46           "count": 1081,
47           "properties": {},
48           "direction": "in",
49           "labels": [
50             "Route"
51           ]
52         },
53       },
54       "type": "node",
55       "properties": {
56         "name": {
57           "existence": false,
58           "type": "STRING",
59           "indexed": true,
60           "unique": true
61         },
62         "country": {
63           "existence": false,
64           "type": "STRING",
65           "indexed": false,
66           "unique": false
67         },
68         "city": {
69           "existence": false,
70           "type": "STRING",
71           "indexed": false,
72           "unique": false
73         }
74       },
75       "labels": [
76         "Airport"
77       ]
78     },
79     "Aircraft": {
80       "count": 114,
81       "relationships": {
82         "USES": {
83           "count": 1437,
84           "properties": {}
85         }
86       },
87       "type": "node",
88       "properties": {
89         "name": {
90           "existence": false,
91           "type": "STRING",
92           "indexed": true,
93           "unique": true
94         },
95         "country": {
96           "existence": false,
97           "type": "STRING",
98           "indexed": false,
99           "unique": false
100        },
101        "direction": "in",
102        "labels": [
103          "Aircraft"
104        ]
105      },
106      "type": "node",
107      "properties": {
108        "name": {
109          "existence": false,
110          "type": "STRING",
111          "indexed": true,
112          "unique": true
113        },
114        "country": {
115          "existence": false,
116          "type": "STRING",
117          "indexed": false,
118          "unique": false
119        },
120        "city": {
121          "existence": false,
122          "type": "STRING",
123          "indexed": false,
124          "unique": false
125        }
126      },
127      "labels": [
128        "Aircraft"
129      ]
130    },
131    "Route": {
132      "count": 57297,
133      "relationships": {
134        "ARRIVES_AT": {
135          "count": 0,
136          "properties": {},
137          "direction": "out",
138          "labels": [
139            "Airport"
140          ]
141        },
142        "DEPARTS_FROM": {
143          "count": 0,
144          "properties": {},
145          "direction": "out",
146          "labels": [
147            "Airport"
148          ]
149        },
150        "USES": {
151          "count": 0,
152          "properties": {},
153          "direction": "out",
154          "labels": [
155            "Aircraft"
156          ]
157        },
158        "OPERATES": {
159          "count": 57297,
160          "properties": {},
161          "direction": "in",
162          "labels": [
163            "Airline"
164          ]
165        }
166      },
167      "type": "node",
168      "properties": {
169        "route_id": {
170          "existence": false,
171          "type": "INTEGER",
172          "indexed": true,
173          "unique": true
174        },
175        "arr_airport": {
176          "existence": false,
177          "type": "STRING",
178          "indexed": false,
179          "unique": false
180        },
181        "airline_name": {
182          "existence": false,
183          "type": "STRING",
184          "indexed": false,
185          "unique": false
186        },
187        "dep_airport": {
188          "existence": false,
189          "type": "STRING",
190          "indexed": false,
191          "unique": false
192        }
193      },
194      "labels": [
195        "Route"
196      ]
197    },
198    "OPERATES": {
199      "type": "relationship",
200      "count": 57297,
201      "properties": {}
202    },
203    "ARRIVES_AT": {
204      "type": "relationship",
205      "count": 1081,
206      "properties": {}
207    },
208    "DEPARTS_FROM": {
209      "type": "relationship",
210      "count": 1081,
211      "properties": {}
212    },
213    "USES": {
214      "type": "relationship",
215      "count": 1437,
216      "properties": {}
217    }
218  }
```

Part 5: Queries

5.1 Performance Analysis

The two most complex queries in terms of time complexity were by far:

- Query 3e which took 17.2 seconds
- Query 4.1 which took 2.2 seconds

All other queries returned in under 0.5 seconds. Speaking purely on the algorithmic side, this graph has 57,296 route nodes and 248,855 total relationships, which is generally considered a small graph by industry standards. In practice, real-world graph databases such as those used by social networks or logistics platforms operate at billions of nodes and edges.

What this suggests is that the dataset for this project is quite small and therefore, time complexity should not really be a factor when considering queries for this assignment. This result also supports the idea that it is unnecessary to drop data for performance sake as suggested in the spec sheet.

5.2 Performance Discussion

This section goes over the performance of the two queries listed above as they are the most substantial. Other queries are ignored as they are trivial. Additionally, I discuss the what if situation if I did opt for the alternate model mentioned in section [1.4 Design Choices](#).

a. Query 3e

[Query 3e](#) performs a variable-length path traversal up to depth 3 (6 relationship hops in the graph model). b is the average branching factor and d is the traversal depth. The worst-case time complexity is therefore:

$$N(d) = b^d$$

However, doubling the number of nodes and edges approximately doubles the branching factor. The resulting increase in search space is given by:

$$\frac{(2b)^d}{b^d} = 2^d = 2^6 = 64$$

This means a graph twice the size would produce roughly [405,000](#) paths which is a [64x](#) increase in worst-case complexity for the same query depth. Deep variable-length path queries therefore do not scale linearly with graph size, and for production-scale aviation graphs, optimisation strategies such as bidirectional BFS or landmark-based shortest path methods would be necessary to maintain acceptable performance.

b. Query 4.1

As for [Query 4.1](#), it performed a standard BFS search which has a time complexity of [O\(V+E\)](#). This scales significantly better than exponential traversal because each node and edge is visited at most once regardless of depth. Doubling the graph size would only double the computation, making BFS-based queries viable even at production scale. This contrasts with Query 3e, where the same doubling produces a 64x increase in search space. Therefore, this suggests that if the data set were slightly larger, some queries such as those like query 3e would become unfeasible while those like query 4a can scale better.

c. Alternate Model

Had I opted for the [\(:Airport\)-\[:ROUTE\]->\(:Airport\)](#) model, each logical hop would be a single relationship traversal rather than two, reducing the worst-case complexity of Q3e from b^6 to b^3 . At production scale this difference could be substantial. However, given that this dataset has only 57,296 routes and 248,855 total relationships, Q3e completes in approximately 17 seconds, so the performance penalty of the [Route-as-node](#) design is acceptable and the modelling benefits for Q3d and airline attribution outweigh the traversal cost.

5.3 Graph Data Science Applications

A natural extension of the BFS shortest path query developed in Section 4.1 is to consider progressively more complex versions of the same routing problem. Each additional constraint transforms the problem into a fundamentally different algorithmic class, each with its own suitable graph algorithm and time complexity.

Level 1: Greedy Weighted Shortest Path (Dijkstra)

The first extension is to assign weights to edges representing estimated route cost, defined as:

$$w(r) = \alpha \cdot d(r) + \beta \cdot f(a) \cdot d(r)$$

where $d(r)$ is the flight distance of route r , $f(a)$ is the fuel consumption rate of aircraft a , and α, β are coefficients weighting distance and fuel cost respectively. The minimum cost path is then:

$$C(p) = \sum_{i=1}^k w(r_i)$$

Dijkstra's algorithm solves this greedily by always expanding the lowest-cost frontier node first. It does not reconsider past decisions, which is valid here because all edge weights are non-negative. Time complexity is $O((V + E) \log V)$ using a priority queue. The business question answered is: what is the cheapest sequence of flights between two airports given fuel costs and aircraft types?

Level 2: Network Flow (Maximum Throughput)

If the problem shifts from minimising cost to maximising passenger throughput across the network, it becomes a maximum flow problem. Each route edge is assigned a capacity representing the maximum number of passengers that can be transported per day based on aircraft size and flight frequency. The question becomes: what is the maximum number of passengers that can be moved from one airport to another through the network simultaneously?

The **Ford-Fulkerson** algorithm solves this by finding augmenting paths from source to sink and incrementally pushing flow along them until no augmenting path remains. Time complexity is $O(VE^2)$ using the **Edmonds-Karp** variant. This is directly applicable to airline hub planning, where an airline needs to determine the maximum passenger capacity between two major hubs across all possible connecting routes.

Level 3: Dynamic Programming (Constrained Optimal Routing)

The most complex variant arises when multiple constraints must be satisfied simultaneously, for example minimising cost while respecting a maximum total flight duration, a maximum number of stops, and specific aircraft availability per leg. This is no longer solvable greedily because a locally cheap edge may lead to a globally infeasible or suboptimal path.

This class of problem is solved with dynamic programming, specifically a variant of the **Bellman-Ford** algorithm which relaxes all edges repeatedly and can handle constraint propagation across the full path. The state space is defined as:

$$dp[v][k] = \text{minimum cost to reach airport } v \text{ in exactly } k \text{ hops}$$

Time complexity is $O(VE)$ per constraint dimension, growing polynomially with the number of added constraints. The business question answered is: what is the optimal flight itinerary from airport A to airport B that minimises total cost, stays within a maximum travel time, and uses only aircraft types available to the operating airline?

5.4 Graph Data Science Discussion

As shown in 5.3, by adding constraints and complexity to query 4a, I was able to show the kind of questions that can be answered which are all relevant to the aviation industry. Each of these questions is genuinely useful for an aviation business when making operational and strategic decisions.

What is notable here is that all three problems are naturally expressed as graph traversal and optimisation problems. The airport-route structure maps directly onto a weighted directed graph, meaning these algorithms can be applied without any schema transformation. This is a key advantage of graph databases over relational models for network-centric domains such as aviation, where the relationships between entities are as analytically important as the entities themselves.

Therefore, **Graph Data Science** has applicable and measurable value in the aviation industry, extending well beyond simple connectivity queries. As the complexity of business questions increases, the graph model remains the most natural and efficient structure for expressing and solving them, making it a compelling choice for any domain where networks, relationships, and traversal are central to the problem. Also note that the algorithmic concepts mentioned in 5.3 were introduced to me in **CITS3001: Advanced Algorithms**.

5.5 Importance of Metadata in Schema Design

Metadata in a graph database refers to data about the structure of the graph itself, including node labels, relationship types, and property keys, rather than the actual data stored within nodes and relationships [4]. In Neo4j, this metadata is accessible via tools such as `apoc.meta.schema()` and `db.schema.visualization()`. Metadata is a critical consideration during schema design for several reasons:

- a) **Structural consistency:** Defining constraints on node labels and properties (as done in Section 2.5 with **CREATE CONSTRAINT**) enforces uniqueness and prevents duplicate nodes during bulk imports. Without these constraints, **MERGE** operations would not function correctly and the graph would contain duplicate entities, directly undermining the data quality work performed during **ETL**.
- b) **Query guidance:** Before writing a Cypher query, a developer must know what labels and relationship types exist in the graph. In a large or unfamiliar database, `apoc.meta.schema()` provides this structural overview instantly, removing the need to manually inspect individual nodes. This is particularly valuable in collaborative environments where multiple developers work on the same graph database.
- c) **Documentation and governance:** In data warehouse contexts, metadata catalogues are a standard component of data management [4]. A well-defined graph schema serves the same purpose, communicating the intended structure of the data to stakeholders without requiring them to query the raw graph directly.

5.6 Leveraging Metadata for Effective Cypher Queries

A developer or data engineer can leverage graph metadata in several concrete ways to write more effective and efficient Cypher queries.

- a) **Schema inspection before querying:** Running `CALL apoc.meta.schema()` or `CALL db.schema.visualization()` before writing queries reveals all available labels, relationship types, and property keys. This prevents errors from mistyped labels or nonexistent properties, which in Cypher would silently return empty results rather than throwing an error [2].
- b) **Index and constraint awareness:** Neo4j maintains metadata about indexes and constraints, accessible via `SHOW INDEXES` and `SHOW CONSTRAINTS`. Queries that filter on indexed properties (such as `Airport.name` with a uniqueness constraint) are significantly faster than those filtering on unindexed properties, as Neo4j can use index lookups instead of full node scans [2]. For example, all queries in this project that match on `Airport {name: ...}` benefit directly from the uniqueness constraint created in Section 2.5.
- c) **Label-based filtering:** Using node labels explicitly in `MATCH` clauses like `MATCH (a:Airport)` allows Neo4j to restrict its search to a specific subset of nodes rather than scanning all 60,694 nodes in the database. This is a direct application of schema metadata as the label system is itself a metadata construct that partitions the node space.

This is an example using `apoc.meta.schema()` to verify structure. It was used in Section 4.2, confirmed that all four node labels and four relationship types were correctly created post-import, and that property keys matched expectations. This is a practical example of metadata-driven query development.

```
CALL apoc.meta.schema()  
YIELD value  
RETURN value;
```

Part 6: Appendix

6.1 AI Declaration

I did this project with the help of Claude (Anthropic, claude.ai) as a generative AI tool throughout the design, implementation, and report writing process. Specifically, I used it for:

- Identifying data quality issues in the raw dataset (section 2.2)
- Designing the ETL Python script (etl.py)
- Debugging Cypher queries for both the required questions and self-designed queries
- Drafting and structuring sections of this report

All AI suggestions were critically evaluated before acceptance. The graph model design decisions, query logic verification, and final judgements on correctness were made by me.

6.2 References

- [1] TU Graz ISDS, "Airports and Airlines Dataset," *GitHub*, 2023. [Online]. Available: https://github.com/tugraz-isds/datasets/tree/master/airports_airlines
- [2] Neo4j, Inc., "Neo4j Graph Data Platform Documentation," 2024. [Online]. Available: <https://neo4j.com/docs/>
- [3] Neo4j, Inc., "APOC Library Documentation," 2024. [Online]. Available: <https://neo4j.com/docs/apoc/current/>
- [4] W. H. Inmon, *Building the Data Warehouse*, 4th ed. Indianapolis, IN: Wiley, 2005.